Atelier Devoxx France 2026

LangChain4j-CDI

IA générative nativement injectable dans Jakarta EE — Construisez des agents IA prêts pour la production avec CDI, MicroProfile et LangChain4j sur WildFly.

Yann Blazart
Tech Lead @ SCIAM

Emmanuel Hugonnet
WildFly Core @ IBM

Ce que vous allez construire

Dans cet atelier pratique, vous allez progressivement construire cinq applications alimentées par l'IA en utilisant **LangChain4j-CDI**, transformant des interfaces Jakarta EE en agents IA injectables avec une seule annotation.

Exercice 1 : Agent IA

Créez un skald viking injectable avec `@RegisterAIService`, des réponses en streaming et de la vision multimodale.

Exercice 2 : Résilience et observabilité

Ajoutez MicroProfile Fault Tolerance (`@Retry` , `@Timeout` , `@CircuitBreaker`) et OpenTelemetry.

🔌

Exercice 3 : Intégration MCP

Connectez votre agent IA à des outils externes via le Model Context Protocol (MCP) — « JDBC pour l'IA ».

🔒

Exercice 4 : Guardrails

Ajoutez des garde-fous sur l'entrée et la sortie de votre skald viking : langue française obligatoire et sans races fantastiques.

🤝

Exercice 5 : A2A Protocol

Orchestrez plusieurs agents IA via le protocole Agent-to-Agent : un writer, un scorer, et un orchestrateur avec boucle de révision.

💡 **FORMAT DE L'ATELIER**

Chaque exercice dispose d'un module `base/` (squelette avec TODOs) et d'un module `solution/` (référence complète). Travaillez dans `base/` et suivez les étapes. Si vous êtes bloqué, la `solution/` est toujours là comme filet de sécurité.

Prérequis

Logiciels requis

OUTIL	VERSION	UTILITÉ
JDK	21+	Développement Java
Maven	3.9+	Outil de build
IDE	Apache NetBeans / IntelliJ IDEA / VS Code	Éditeur de code
curl	n'importe quelle	Test d'API
Clé API Mistral AI *ou* **Ollama**	—	Fournisseur LLM — distant (Mistral AI, gratuit) ou local (Ollama)

OUTIL	VERSION	UTILITÉ
Docker / Podman	n'importe quelle	Stack Grafana LGTM (Exercice 2 uniquement)

1. Récupérer le code source

```
git clone https://github.com/ehsavoie/langchain4j-cdi-lab-2026.git
cd langchain4j-cdi-lab-2026/demo-project
```

2. Choisir votre fournisseur LLM

🎲 **DEUX OPTIONS AU CHOIX**

Option A — Mistral AI (distant) : inscrivez-vous gratuitement sur console.mistral.ai pour obtenir une clé API. Aucune installation locale requise.

Option B — Ollama (local) : installez Ollama, puis téléchargez le modèle :

```
# Dans un premier terminal (laisser tourner) :
ollama serve

# Dans un second terminal :
ollama pull ministral-3:3b
```

Ollama doit rester actif sur `http://localhost:11434` pendant tout l'atelier.

⚙ **COMMENT CONFIGURER LE FOURNISSEUR DANS CHAQUE EXERCICE**

Chaque module contient un fichier `microprofile-config.properties` structuré ainsi :

```
# ---- Option A : Mistral AI (distant) ----
dev.langchain4j.cdi.plugin.my-model.class=...MistralAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=mistral-small-latest

# ---- Option B : Ollama (local) ----
# dev.langchain4j.cdi.plugin.my-model.class=...OllamaChatModel
# dev.langchain4j.cdi.plugin.my-model.config.base-url=http://localhost:11434
# dev.langchain4j.cdi.plugin.my-model.config.model-name=ministral-3:3b
```

```
dev.langchain4j.cdi.plugin.my-model.config.temperature=0.9
dev.langchain4j.cdi.plugin.my-model.config.log-requests=true
```

Par défaut, l'Option A (Mistral AI) est active. Pour passer à Ollama :

1. **Commentez** les 3 lignes de l'Option A (ajoutez `#` devant)
2. **Décommentez** les 3 lignes de l'Option B (retirez les `#`)

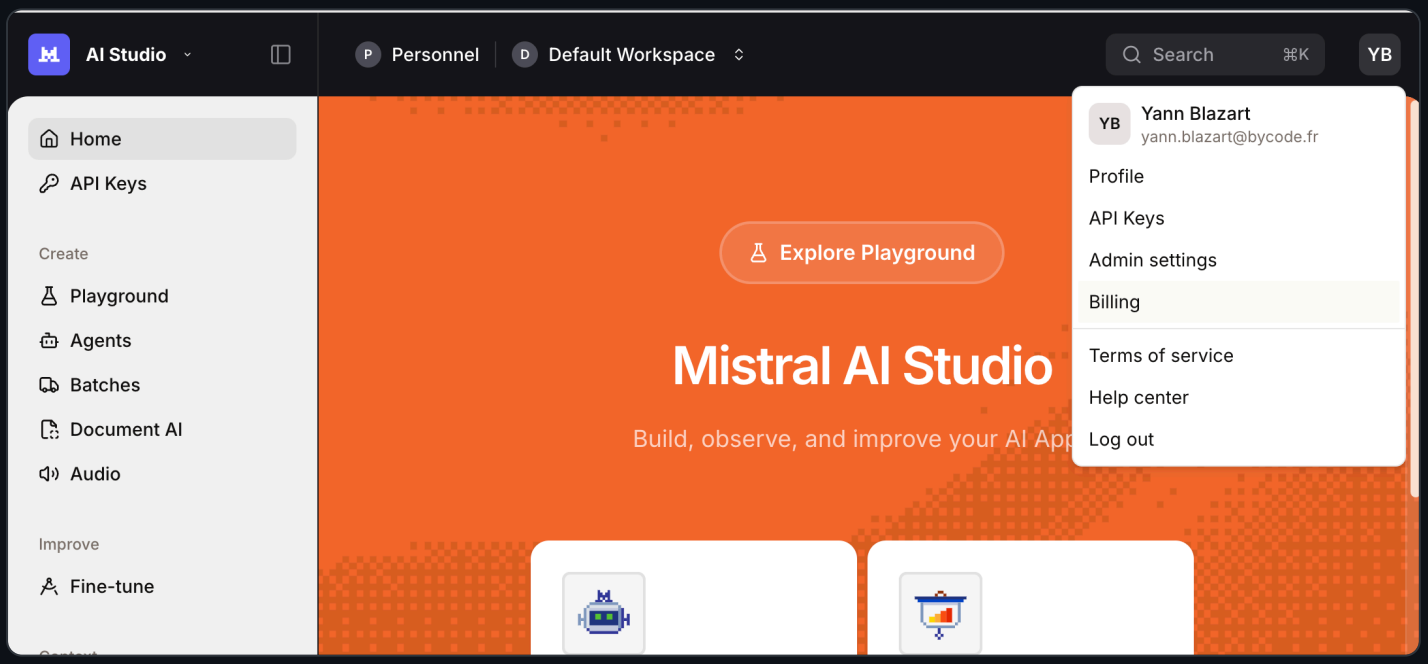
Les propriétés communes (`temperature`, `log-requests`, etc.) sont en dessous et **restent inchangées** — elles s'appliquent quel que soit le fournisseur choisi.

Option A : Configurer Mistral AI (distant)

Inscrivez-vous sur console.mistral.ai et suivez ces étapes pour obtenir une clé API gratuite :

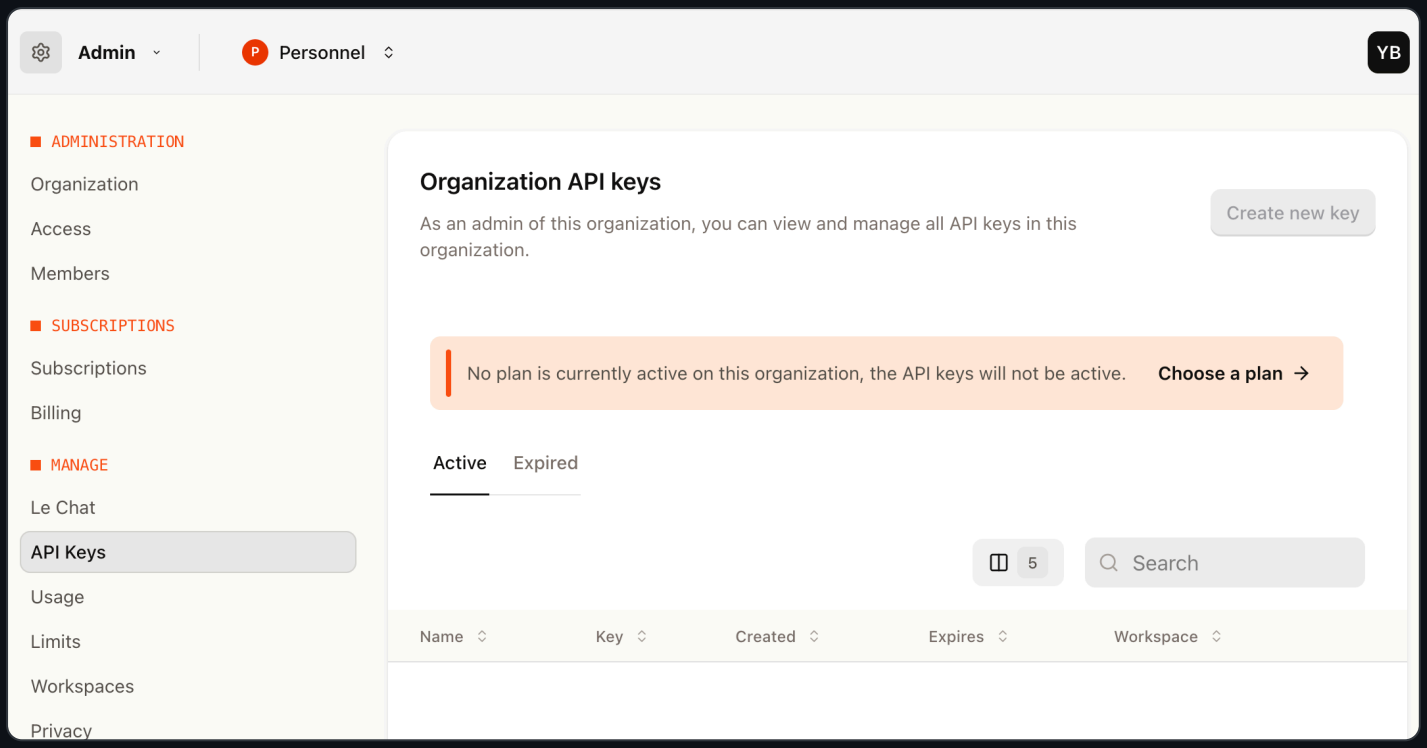
Étape 1 : Accéder aux clés API

Depuis la page d'accueil de Mistral AI Studio, cliquez sur l'icône de votre profil (coin supérieur droit), puis sélectionnez **API Keys**.



Étape 2 : Choisir un forfait

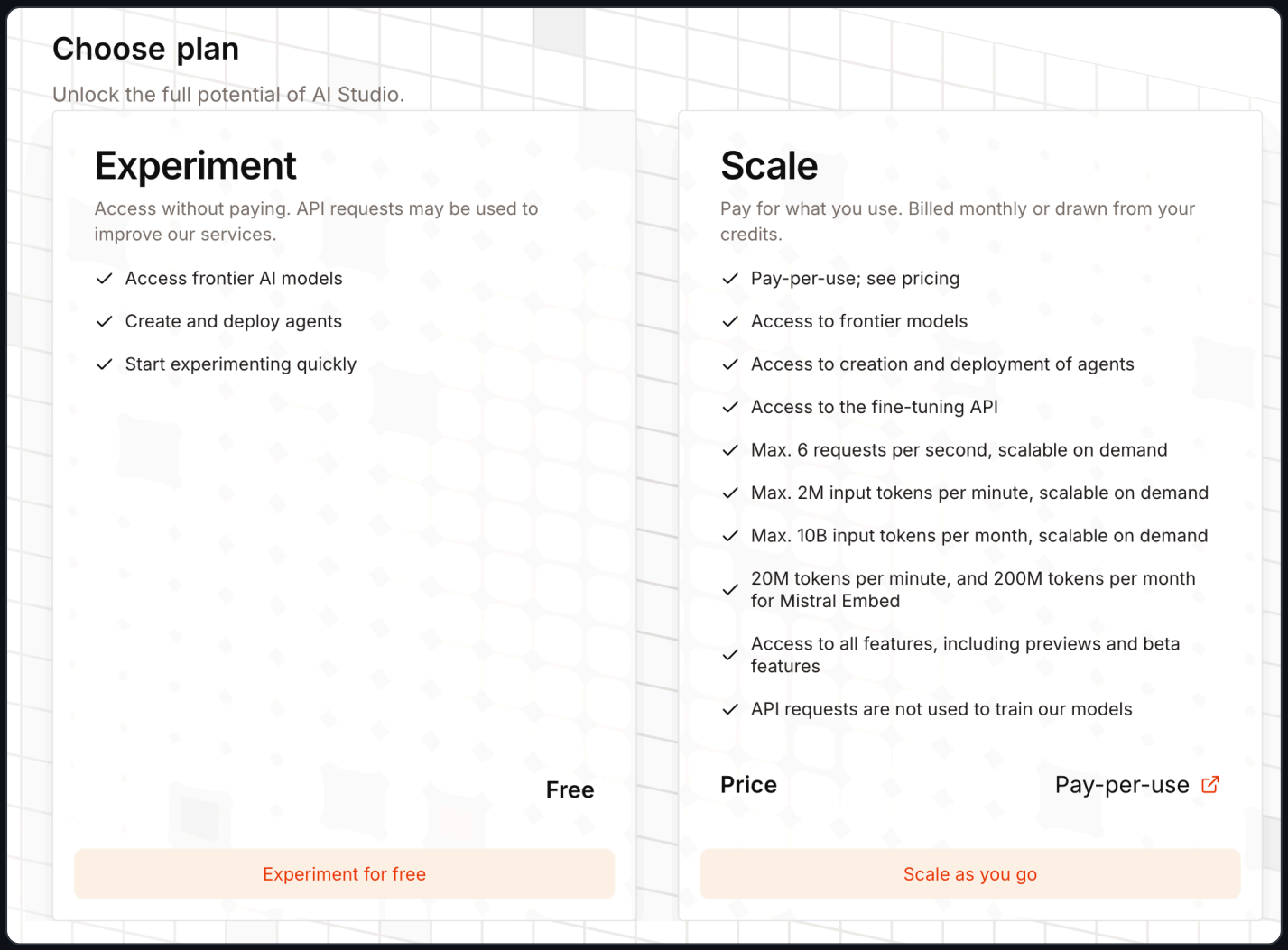
Sur la page API Keys, vous verrez un avertissement : *"No plan is currently active on this organization, the API keys will not be active."* Cliquez sur **Choose a plan** pour activer vos clés.



> ⚠️ **IMPORTANT**
>
> Sans forfait actif, vos clés API seront créées mais **ne fonctionneront pas**. Vous obtiendrez une erreur `Unauthorized` lors de l'appel de l'API.

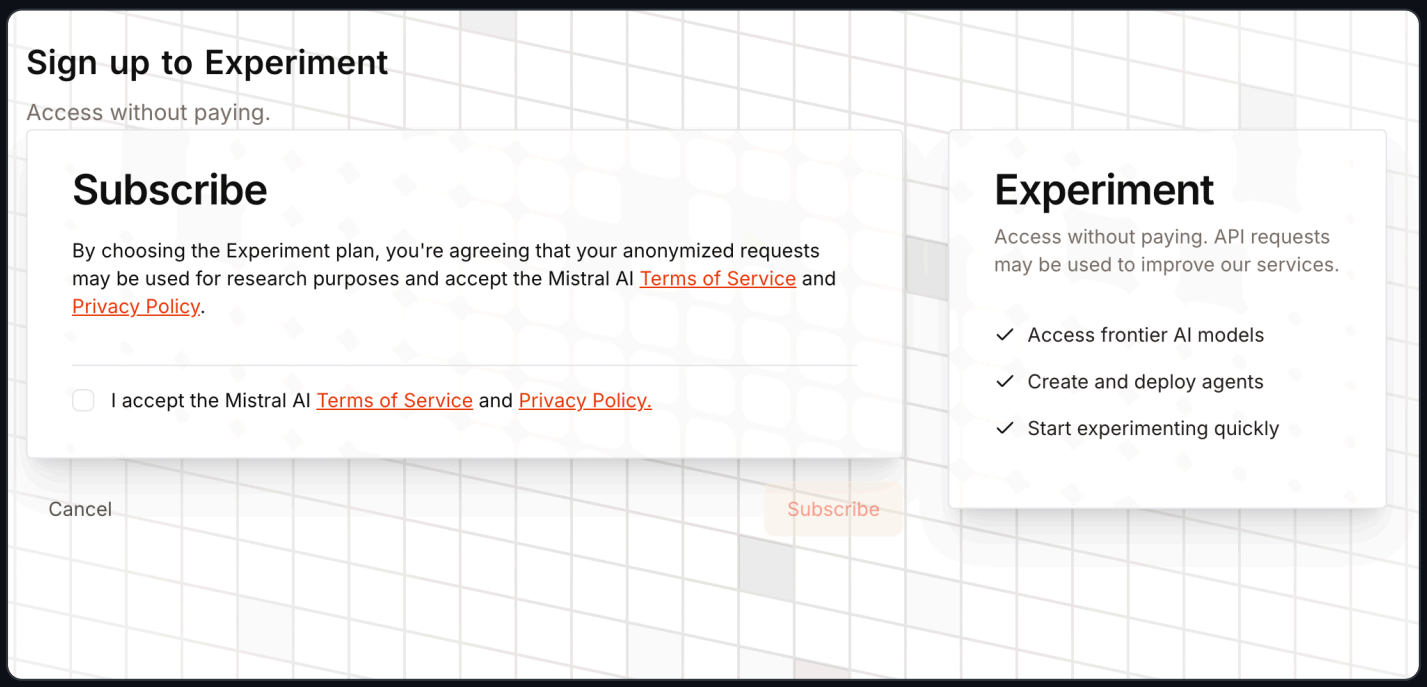
Étape 3 : Sélectionner le forfait gratuit "Experiment"

Choisissez le forfait **Experiment** (côté gauche) — il est complètement **gratuit** et vous donne accès à tous les modèles d'IA de pointe. Cliquez sur **"Experiment for free"**.



Étape 4 : Accepter les conditions et s'abonner

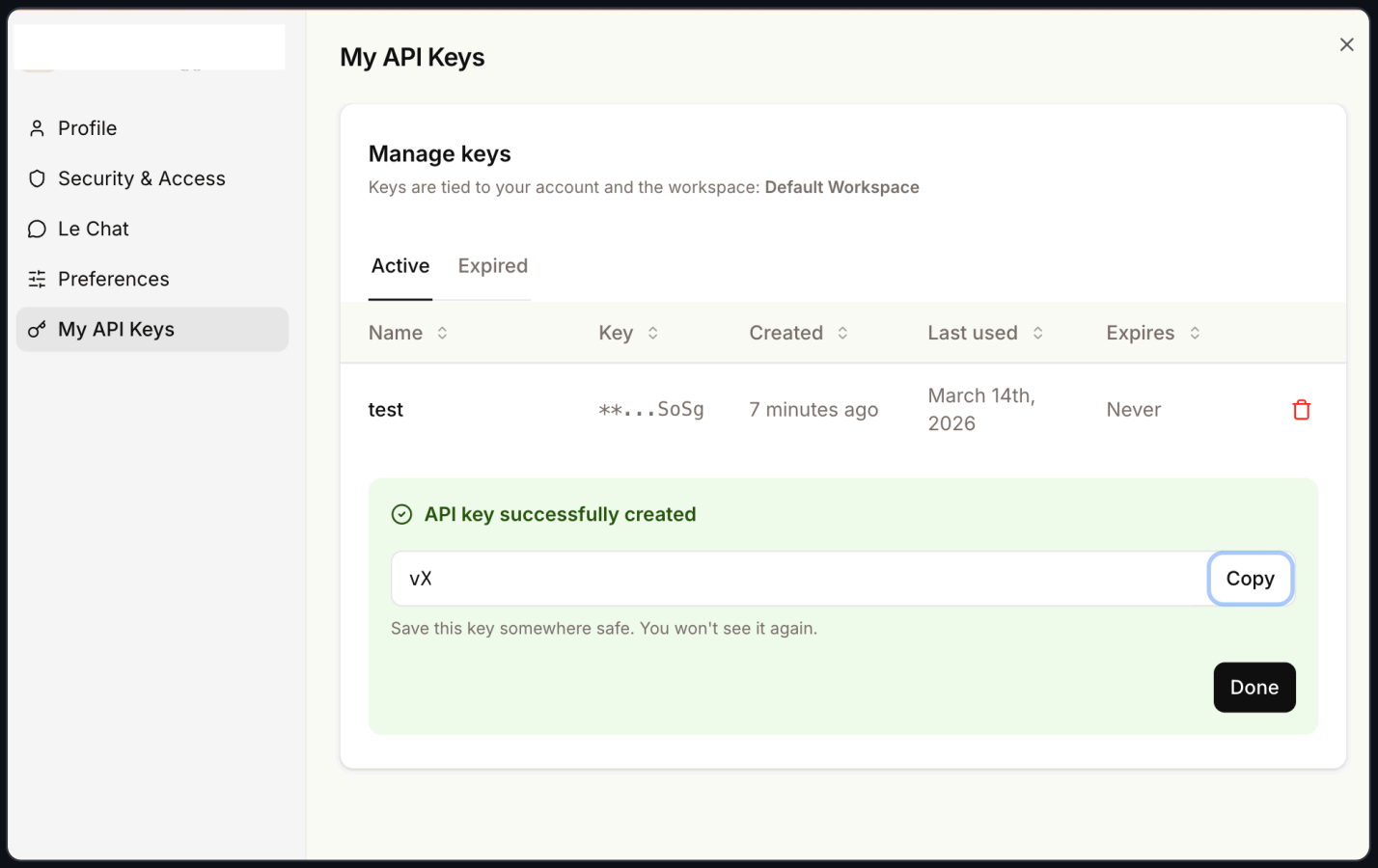
Cochez la case pour accepter les conditions d'utilisation et la politique de confidentialité, puis cliquez sur **Subscribe**.



Il vous sera ensuite demandé de fournir votre **numéro de téléphone** pour recevoir un **code d'activation par SMS**. Ceci est requis pour activer le forfait gratuit Experiment.

Étape 5 : Créer et exporter votre clé API

Une fois votre forfait activé, retournez sur la page API Keys et cliquez sur **"Create new key"**. Copiez la clé immédiatement — vous ne pourrez plus la revoir.



Puis exportez-la dans votre terminal :

```
# Linux / macOS
export MISTRAL_API_KEY=your-api-key-here

# Windows (PowerShell)
$env:MISTRAL_API_KEY="your-api-key-here"
```

> 💡 **OPTION B : OLLAMA LOCAL**
>
> Si vous avez choisi Ollama, **ignorez toute la section Mistral AI ci-dessus**. Pas besoin de clé API — il suffit de commenter/décommenter les blocs dans `microprofile-config.properties` comme expliqué plus haut.

3. WildFly (automatique)

WildFly 39 est automatiquement téléchargé et provisionné par le plugin Maven. Aucune installation manuelle nécessaire :

```
# Ceci télécharge WildFly, le démarre et déploie l'application
cd demo-1-ai-agent/solution
mvn clean wildfly:dev
```

L'objectif `wildfly:dev` est votre meilleur ami — il recharge à chaud à chaque `mvn package`.

3bis. Configurer le fournisseur LLM dans la solution

Avant de vérifier votre installation, assurez-vous que le fichier `demo-1-ai-agent/solution/src/main/resources/META-INF/microprofile-config.properties` est configuré pour votre fournisseur :

- **Option A — Mistral AI (distant)** : c'est la configuration par défaut. Vérifiez simplement que la variable `MISTRAL_API_KEY` est bien exportée dans votre terminal.
- **Option B — Ollama (local)** : pour **chaque bloc** (my-model, my-streaming-model, vision-model), commentez les 3 lignes de l'Option A et décommentez les 3 lignes de l'Option B.

```
# Exemple pour my-model — Option B : Ollama
# (commentez les 3 lignes Mistral AI ci-dessus, puis décommentez celles-ci)
dev.langchain4j.cdi.plugin.my-model.class=dev.langchain4j.model.ollama.OllamaChatModel
dev.langchain4j.cdi.plugin.my-model.config.base-url=http://localhost:11434
dev.langchain4j.cdi.plugin.my-model.config.model-name=ministral-3:3b
```

> ⚠ **ATTENTION**
>
> Le fichier contient **3 modèles** (my-model, my-streaming-model, vision-model). Pensez à basculer **les trois** si vous utilisez Ollama.

4. Vérifier votre installation

```
# Compiler tous les modules pour télécharger les dépendances
cd demo-project
mvn clean install -DskipTests

# Test rapide de vérification
cd demo-1-ai-agent/solution
```

```
mvn clean wildfly:dev

# Dans un autre terminal :
curl -X POST -H "Content-Type: text/plain" \
  -d "Sing me a heroic song" \
  http://localhost:8080/demo-1/api/chat
```

Vous pouvez aussi tester directement dans votre navigateur en ouvrant http://localhost:8080/demo-1 — l'interface web vous permet de discuter avec le skald viking.

> **✓ PRÊT !**
>
> Si vous obtenez une réponse du skald viking IA (via curl ou via le navigateur), votre environnement est correctement configuré. Arrêtez le serveur (`Ctrl+C`) et passez aux exercices.

Vue d'ensemble de l'architecture

LangChain4j-CDI est une extension CDI qui fait le pont entre **LangChain4j** (le framework IA Java) et les serveurs **Jakarta EE / MicroProfile**. Elle transforme des interfaces annotées en agents IA injectables.

Comment ça fonctionne

1. Scan
L'extension CDI scanne les interfaces `@RegisterAIService` au démarrage

2. Lire les attributs
Pour chaque interface : `chatModelName`, `tools`, `chatMemoryProviderName`, `contentRetrieverName`

3. Créer un bean synthétique
Un proxy LangChain4j est enveloppé dans un bean CDI géré

4. Résoudre les composants via SPI
Le SPI `LLMConfig` lit depuis MicroProfile Config : `dev.langchain4j.cdi.plugin.<name>.class=...`

5. Injecter !
`@Inject Assistant assistant;` — ça fonctionne tout simplement

L'avant / après

Sans CDI (verbeux)

```
ChatLanguageModel model =
  MistralAiChatModel.builder()
    .apiKey(System.getenv("MISTRAL_API_KEY"))
    .modelName("pixtral-large-latest")
    .temperature(0.3)
    .maxTokens(1000)
    .build();

ChatMemory memory =
  MessageWindowChatMemory.builder()
    .maxMessages(20).build();

Assistant assistant =
  AiServices.builder(Assistant.class)
    .chatLanguageModel(model)
    .chatMemory(memory)
    .tools(new VikingTools())
    .build();
```

Avec CDI (propre)

```
@RegisterAIService(
  chatModelName = "my-model",
  tools = VikingTools.class)
public interface Assistant {

  @SystemMessage("You are a
    Viking expedition guide")
  String chat(
    @UserMessage String message);
}

// In your REST endpoint:
@Inject
Assistant assistant;
```

Double support d'extension CDI

LangChain4j-CDI fournit **les deux** types d'extensions, donc ça fonctionne sur **n'importe quel** serveur CDI :

TYPE D'EXTENSION	TRAITEMENT	SERVEURS
Portable Extension	Runtime	WildFly, Payara, GlassFish, Liberty
Build Compatible Extension	Compile-time	Quarkus, Helidon

Structure du projet

```
demo-project/
├── pom.xml ← Parent POM (centralized versions)
├── demo-1-ai-agent/ ← Injectable AI Agent
│   ├── base/ ← Skeleton with TODOs (your workspace)
│   └── solution/ ← Complete reference
```

```
├── demo-2-ft-telemetry/ ← Memory + RAG + Tools + FT + Telemetry
│   ├── base/
│   └── solution/
├── demo-3-mcp/ ← MCP Integration
│   ├── mcp-server/ ← Serveur MCP standalone (Helidon 4)
│   ├── base/
│   └── solution/
└── demo-4-guardrails/ ← Input & Output Guardrails
    ├── base/ ← Squelette avec TODOs (votre espace de travail)
    └── solution/
```

Diapositives de présentation

L'atelier comprend un jeu de diapositives Reveal.js couvrant la théorie derrière chaque exercice. Voici ce que couvre chaque section.

> 🎬 **LANCER LES DIAPOSITIVES**
>
> Ouvrir directement `slides/index.html` dans le navigateur. Appuyez sur S pour les notes du présentateur, F pour le plein écran, 0 pour la vue d'ensemble.

Introduction et contexte

La présentation commence par l'énoncé du problème : LangChain4j fonctionne très bien avec Quarkus, mais les entreprises utilisent **WildFly, Payara, Liberty, GlassFish, Helidon**. La réponse : `@RegisterAIService` — « Injectez un agent IA aussi naturellement qu'un EntityManager ».

Internes de l'extension CDI

Plongée approfondie dans la façon dont l'extension CDI scanne `@RegisterAIService`, crée des beans synthétiques, résout les composants via le SPI `LLMConfig` et fait le pont vers MicroProfile Config. Couvre à la fois les extensions Portable et Build Compatible.

Fonctionnalités progressives

- **Mémoire de conversation** : `@MemoryId` + `ChatMemoryProvider` pour l'état par session
- **RAG** : `contentRetrieverName` avec embeddings et vector stores
- **Outils / Function Calling** : beans CDI avec méthodes `@Tool` — la fonctionnalité majeure
- **MCP** : Model Context Protocol via `toolProviderName` — « JDBC pour l'IA »

Prêt pour la production

- **MicroProfile Fault Tolerance** : `@Retry`, `@Timeout`, `@Fallback`, `@CircuitBreaker`
- **MicroProfile Telemetry** : traces et métriques OpenTelemetry avec Grafana LGTM
- **Modules LangChain4j-CDI** : core, portable-ext, build-compatible-ext, fault-tolerance, telemetry
- **A2A (Agent-to-Agent)** : aperçu du prochain protocole pour la communication inter-agents

Exercice 1 : Agent IA injectable

Message clé : « De 15 lignes de code passe-partout à 1 annotation + 1 propriété de configuration »

📌 **OBJECTIF**

Créez un chatbot IA simple en utilisant `@RegisterAIService`, injectez-le dans un point d'accès JAX-RS et testez-le. Ensuite, ajoutez des capacités de streaming et de vision multimodale.

```
cd demo-project/demo-1-ai-agent/base
# Ouvrez ce répertoire dans votre IDE
```

Fichiers clés

FICHIER	UTILITÉ
`ChatAssistant.java`	Interface du service IA (votre tâche principale)
`ChatAssistantStreaming.java`	Service IA en streaming
`ChatResource.java`	Point d'accès REST JAX-RS
`ImageAnalyzerServlet.java`	Servlet de vision multimodale
`microprofile-config.properties`	Configuration LLM

1 Créer l'interface du service IA

Ouvrez `ChatAssistant.java` . C'est une interface Java simple. Votre tâche : la transformer en agent IA injectable.

Ajouter @RegisterAIService

```
@RegisterAIService(chatModelName = "my-model")
public interface ChatAssistant {

    @SystemMessage("""
        You are a Viking skald (bard) entertaining warriors in a great hall.
        You tell epic tales, sing heroic songs, compose witty verses about
        battles, voyages, and the glorious chaos of Viking life.
        Your style is bold, dramatic, and full of Norse spirit.
        """)
    String chat(@UserMessage String userMessage);
}
```

💡 **CE QUI SE PASSE**

- `@RegisterAIService` indique à l'extension CDI de créer un bean injectable pour cette interface
- `chatModelName = "my-model"` fait référence à la configuration du modèle dans `microprofile-config.properties`
- `@SystemMessage` définit la personnalité et le comportement de l'IA
- `@UserMessage` marque le paramètre comme entrée utilisateur

2 Câbler le point d'accès REST

Ouvrez `ChatResource.java` . Injectez le service IA et appelez-le :

```
@Path("/chat")
@ApplicationScoped
public class ChatResource {

    @Inject
    ChatAssistant assistant;  // CDI injection!

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String chat(String message) {
```

```
        return assistant.chat(message);
    }
}
```

✓ **POINT CLÉ**

Le code métier n'a **aucune dépendance sur LangChain4j**. Le seul import est votre propre interface. Changer de framework IA demain ? Changez l'implémentation, pas le code métier.

3 Configurer le modèle

Ouvrez `src/main/resources/META-INF/microprofile-config.properties` et décommentez/ajoutez la configuration du modèle :

```
# Chat model (Mistral AI)
dev.langchain4j.cdi.plugin.my-model.class=\
    dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=mistral-small-latest
dev.langchain4j.cdi.plugin.my-model.config.temperature=0.9
```

🏠 **OLLAMA LOCAL**

Dans `microprofile-config.properties`, commentez les 3 lignes de l'**Option A** et décommentez les 3 lignes de l'**Option B**. Les propriétés communes (`temperature`, `log-requests`...) restent inchangées.

⚠ **CRITIQUE : LE PRÉFIXE .CONFIG.**

Les propriétés du builder **doivent** utiliser le préfixe `.config.` : `dev.langchain4j.cdi.plugin.<name>.config.<prop>=value`. Sans `.config.`, la propriété est **ignorée silencieusement**.

Déployer et tester

```
mvn clean wildfly:dev
```

```
# Dans un autre terminal :
curl -X POST -H "Content-Type: text/plain" \
  -d "Tell me a saga about Java developers on a Viking voyage" \
  http://localhost:8080/demo-1/api/chat
```

4 Ajouter les réponses en streaming

Ouvrez `ChatAssistantStreaming.java` et créez un service IA en streaming :

```
@RegisterAIService(streamingChatModelName = "my-streaming-model")
public interface ChatAssistantStreaming {

    @SystemMessage("You are a Viking storyteller in a great hall...")
    TokenStream chatStream(@UserMessage String userMessage);
}
```

Puis dans `ChatResource.java` , ajoutez le point d'accès SSE :

```
@Inject
ChatAssistantStreaming streamingAssistant;

@GET
@Path("/stream")
@Produces(MediaType.SERVER_SENT_EVENTS)
public void chatStream(@QueryParam("message") String message,
                       @Context SseEventSink sink, @Context Sse sse) {
    TokenStream tokenStream = streamingAssistant.chatStream(message);

    tokenStream
        .onPartialResponse(token -> 
            sink.send(sse.newEvent("token", token)))
        .onCompleteResponse(resp -> {
            sink.send(sse.newEvent("done", ""));
            sink.close();
        })
        .onError(error -> {
            sink.send(sse.newEvent("error", error.getMessage()));
            sink.close();
        })
        .start();
}
```

Ajoutez la configuration du modèle streaming dans `microprofile-config.properties` :

```
dev.langchain4j.cdi.plugin.my-streaming-model.class=\
    dev.langchain4j.model.mistralai.MistralAiStreamingChatModel
dev.langchain4j.cdi.plugin.my-streaming-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-streaming-model.config.model-name=mistral-small-latest
```

🏠 **OLLAMA LOCAL**

Même principe : commentez l'**Option A** du streaming model et décommentez l'**Option B** dans `microprofile-config.properties`.

5 Vision : analyse d'image

Ouvrez `ImageAnalyzerServlet.java` et injectez un modèle de vision :

```
@WebServlet("/uploadServlet")
@MultipartConfig
public class ImageAnalyzerServlet extends HttpServlet {

    @Inject
    @Named("vision-model")
    ChatModel visionModel;

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) throws ... {
        Part file = request.getPart("file");

        UserMessage userMessage = UserMessage.from(
            TextContent.from("Describe this image in detail"),
            ImageContent.from(encodeBase64(file.getInputStream()),
                              file.getContentType())
        );

        ChatResponse answer = visionModel.chat(userMessage);
        response.getWriter().write(answer.aiMessage().text());
    }
}
```

Et la configuration du modèle de vision :

```
dev.langchain4j.cdi.plugin.vision-model.class=\
    dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.vision-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.vision-model.config.model-name=pixtral-large-latest
```

🏠 **OLLAMA LOCAL**

Même principe : commentez l'**Option A** du vision model et décommentez l'**Option B**. Le modèle Ollama déjà téléchargé (`ministral-3:3b`) sera utilisé.

✓ Tester et valider l'exercice 1

```
# Chat standard
curl -X POST -H "Content-Type: text/plain" \
  -d "What is CDI in 2 sentences?" \
  http://localhost:8080/demo-1/api/chat

# Streaming (ouvrir dans le navigateur)
# http://localhost:8080/demo-1/stream.html

# Analyse d'image (ouvrir dans le navigateur)
# http://localhost:8080/demo-1/image.html

# Vérification de santé
curl http://localhost:8080/demo-1/api/chat/health
```

- ✓ Le chat standard renvoie une réponse du skald viking
- ✓ Le streaming affiche les tokens arrivant en temps réel
- ✓ Le téléchargement d'image renvoie une description générée par l'IA

💡 **BLOQUÉ ?**

Passez à la solution : `cd ../solution && mvn clean wildfly:dev`

Exercice 2 : Tolérance aux pannes et télémétrie

Message clé : « Les annotations MicroProfile fonctionnent sur les services IA parce qu'ils sont des beans CDI »

📌 **OBJECTIF**

Commencez avec un guide d'expéditions viking fonctionnel qui possède déjà la mémoire, RAG et des outils. Votre tâche : ajouter des annotations MicroProfile Fault Tolerance pour le rendre prêt pour la production, puis l'observer avec OpenTelemetry.

```
cd demo-project/demo-2-ft-telemetry/base
mvn clean wildfly:dev
```

Ouvrez votre navigateur vers http://localhost:8080/demo-2 pour tester l'application.

Ce qui fonctionne déjà

Le module base est pré-câblé avec ces fonctionnalités :

🧠

Mémoire de conversation

`@MemoryId` fournit une mémoire par session via `ChatMemoryProviderBean`. Chaque onglet du navigateur obtient sa propre conversation.

📄

RAG

`ExpeditionDetailsProvider` ingeste les données d'expéditions viking dans un vector store en mémoire en utilisant les embeddings Mistral.

🔧

Outils

`VikingTools` est un bean CDI avec 5 méthodes `@Tool` : lister expéditions, enrôler guerrier, annuler, vérifier disponibilité, voir enrôlements.

Testez que tout fonctionne avant d'ajouter la tolérance aux pannes :

```
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-123" \
  -d "What expeditions are available?" \
  http://localhost:8080/demo-2/api/chat
```

1 Ajouter @Retry

Ouvrez `ChatAssistant.java`. L'interface du service IA possède déjà `@RegisterAIService` avec la mémoire, le RAG et les outils. Ajoutez l'annotation `@Retry` :

```
import org.eclipse.microprofile.faulttolerance.Retry;

@RegisterAIService(chatModelName = "my-model",
                   chatMemoryProviderName = "my-memory",
                   contentRetrieverName = "my-rag",
                   tools = VikingTools.class)
public interface ChatAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    String chat(@MemoryId String sessionId,
                @UserMessage String message);
}
```

> 💡 **POURQUOI ÇA FONCTIONNE**
>
> Comme `@RegisterAIService` crée un bean CDI, **tous les intercepteurs MicroProfile s'appliquent**. Le conteneur CDI enveloppe le proxy avec l'intercepteur Fault Tolerance — aucun code de liaison nécessaire.

2 Ajouter @Timeout et @Fallback

Empilez les annotations Fault Tolerance :

```
import org.eclipse.microprofile.faulttolerance.*;
import java.time.temporal.ChronoUnit;

@RegisterAIService(chatModelName = "my-model",
                   chatMemoryProviderName = "my-memory",
                   contentRetrieverName = "my-rag",
```

```
                    tools = VikingTools.class)
public interface ChatAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    @Timeout(value = 30, unit = ChronoUnit.SECONDS)
    @Fallback(fallbackMethod = "chatFallback")
    String chat(@MemoryId String sessionId,
                @UserMessage String message);

    default String chatFallback(String sessionId, String message) {
        return "I'm sorry, our AI service is temporarily unavailable. "
             + "Please try again in a moment. "
             + "Our team has been notified.";
    }
}
```

3 Ajouter @CircuitBreaker

Ajoutez la dernière pièce — le circuit breaker empêche de surcharger un service défaillant :

```
@Retry(maxRetries = 3, delay = 1000)
@Timeout(value = 30, unit = ChronoUnit.SECONDS)
@Fallback(fallbackMethod = "chatFallback")
@CircuitBreaker(requestVolumeThreshold = 5, failureRatio = 0.5)
String chat(@MemoryId String sessionId,
            @UserMessage String message);
```

Rechargement à chaud : sauvegardez et exécutez `mvn package` (ou laissez votre IDE compiler automatiquement).

✓ TESTER LA RÉSILIENCE

Simulez une panne : coupez le wifi pour simuler un souci réseau, puis envoyez des requêtes. Si vous utilisez Ollama, coupez simplement le serveur. Vous devriez voir les tentatives de retry, puis le message de fallback. Rétablissez la connexion (ou relancez Ollama) et le circuit breaker se rétablit automatiquement.

4 Ajouter l'observabilité OpenTelemetry

Démarrez la stack Grafana LGTM (Loki + Grafana + Tempo + Mimir) :

```
# Démarrer la stack LGTM complète (Loki + Grafana + Tempo + Mimir)
# avec le collecteur OpenTelemetry intégré
podman run -p 3000:3000 -p 4317:4317 -p 4318:4318 --rm -ti grafana/otel-lgtm

# Ou avec Docker
docker run -p 3000:3000 -p 4317:4317 -p 4318:4318 --rm -ti grafana/otel-lgtm
```

La configuration de télémétrie est déjà dans `microprofile-config.properties` :

```
# OpenTelemetry
otel.exporter.otlp.endpoint=http://localhost:4318
otel.service.name=demo-2-langchain4j-cdi
otel.traces.exporter=otlp
otel.metrics.exporter=otlp

# LangChain4j-CDI Telemetry Listeners
dev.langchain4j.cdi.plugin.my-model.config.listeners=\
    dev.langchain4j.cdi.telemetry.SpanChatModelListener,\
    dev.langchain4j.cdi.telemetry.MetricsChatModelListener
```

Envoyez quelques requêtes de chat, puis ouvrez http://localhost:3000 (Grafana). Naviguez vers **Explore** → **Tempo** et recherchez le service `demo-2-langchain4j-cdi` .

Vous verrez des traces distribuées montrant :

- La latence des appels LLM
- Le nombre de tokens (entrée/sortie)
- Les appels d'outils et leur durée
- Les étapes de récupération RAG

✓ Tester et valider l'exercice 2

```
# Chat avec session
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-456" \
  -d "What expeditions are available?" \
  http://localhost:8080/demo-2/api/chat
```

```
# Enrôler un guerrier
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-456" \
  -d "Enroll Erik Bloodaxe for the first expedition" \
  http://localhost:8080/demo-2/api/chat

# Inspecter la mémoire
curl "http://localhost:8080/demo-2/api/chat/memory?sessionId=test-456"
```

- ✓ La mémoire conserve le contexte entre les messages d'une même session
- ✓ Le RAG récupère les informations pertinentes sur les expéditions
- ✓ Les outils exécutent les enrôlements, annulations et requêtes
- ✓ Le fallback répond gracieusement quand l'IA est indisponible
- ✓ Les traces apparaissent dans Grafana/Tempo

Exercice 3 : Intégration MCP

Message clé : « MCP est le JDBC de l'IA — vos agents communiquent avec n'importe quel serveur d'outils »

> 📌 **OBJECTIF**
>
> Connectez un agent IA à un serveur MCP (Model Context Protocol) externe. Le serveur MCP fournit des outils de lancer de runes, et votre agent IA anime un jeu de Hnefatafl au Grand Thing via le protocole.

Qu'est-ce que MCP ?

Le **Model Context Protocol** est un standard ouvert pour connecter les agents IA à des outils externes. Voyez-le comme un adaptateur universel : un seul protocole, n'importe quel serveur d'outils, n'importe quel langage.

```
🚀 Agent CDI      →   🔌 McpToolProvider    →   🌐 MCP Server (any language)
@RegisterAIService      toolProviderName          JSON-RPC via Streamable HTTP
```

0 Compiler et démarrer le serveur MCP

D'abord, compilez et démarrez le serveur MCP de dés externe :

```
# Compiler et démarrer le serveur MCP avec Helidon 4 (écoute sur le port 8090)
cd demo-project/demo-3-mcp/mcp-server
mvn clean package
java -jar target/casino-dice-roller.jar
```

Le serveur MCP est une application MicroProfile propulsée par **LangChain4j-CDI MCP Server** et **Helidon 4**. Il expose automatiquement un endpoint `/mcp` qui parle le protocole MCP (JSON-RPC 2.0 sur HTTP Streamable). L'outil `roll` génère des résultats de dés aléatoires pour le jeu viking.

> 💡 **LAISSEZ-LE TOURNER**
>
> Laissez le serveur MCP tourner dans un terminal séparé. L'agent IA s'y connectera via l'endpoint MCP sur `localhost:8090/mcp`.

Tester avec MCP Inspector

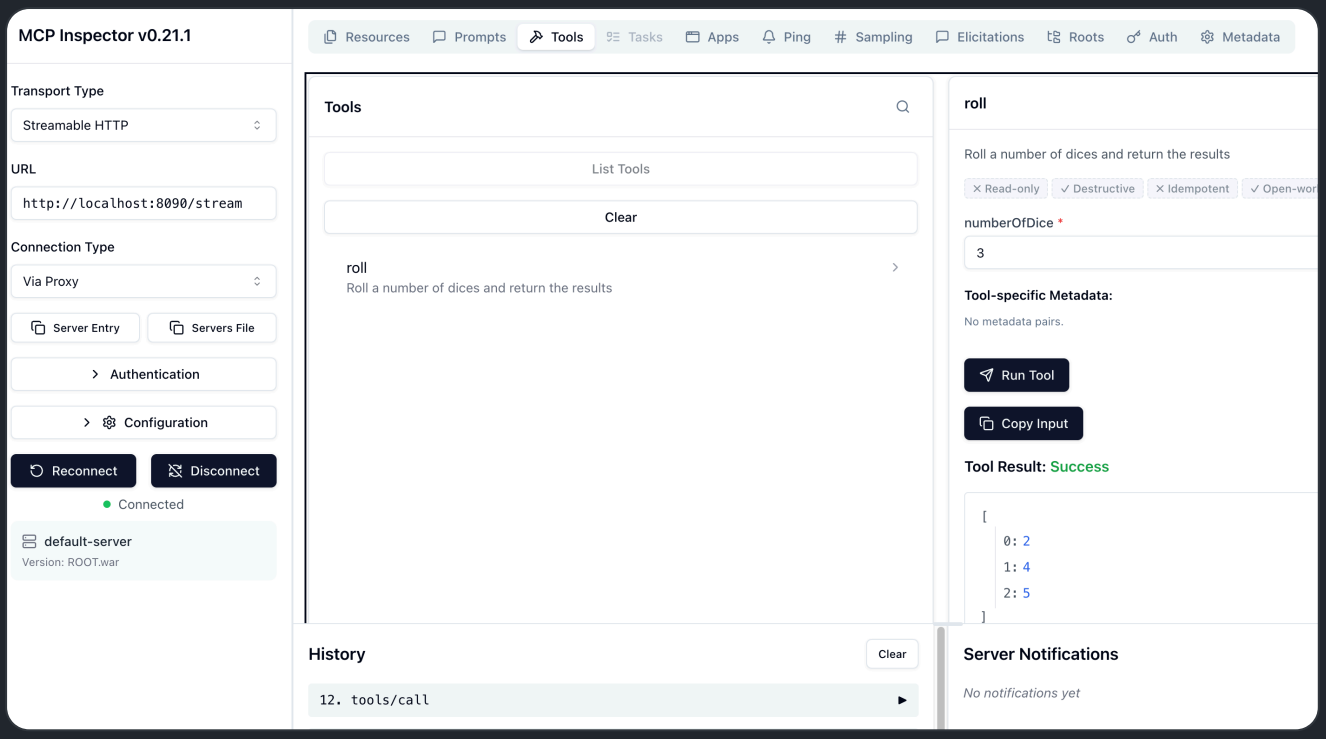
Avant de connecter le serveur MCP à l'agent IA, vous pouvez vérifier qu'il fonctionne en utilisant le **MCP Inspector** — un outil de débogage visuel pour les serveurs MCP. Dans un nouveau terminal :

```
# Lancer le MCP Inspector
npx @modelcontextprotocol/inspector
```

Ceci ouvre une interface web (généralement sur `http://localhost:6274`). Configurez la connexion :

- **Transport Type** : `Streamable HTTP`
- **URL** : `http://localhost:8090/mcp`

Cliquez sur **Connect**, puis sur l'onglet **Tools** et sur l'outil `roll` pour l'inspecter et le tester interactivement :



🛠️ POINTS À VÉRIFIER

- Le statut de connexion affiche **Connected**
- L'outil `roll` apparaît dans la liste des Tools
- Mettez `numberOfDice` à `2`, cliquez sur **Run Tool**, et vérifiez que vous obtenez des résultats de dés

1 Créer le service IA du Jarl du Grand Thing

Ouvrez `HnefataflJarlAI.java` dans `demo-3-mcp/base/` :

```
@RegisterAIService(chatModelName = "mistral",
                   toolProviderName = "mcp")
public interface HnefataflJarlAI {

    @SystemMessage("""
        You are Ragnar the Skald, the Jarl hosting Hnefatafl
        at the Grand Thing of the Northern warriors.

        RULES: Cast 2 rune stones with roll(numberOfDice=2).
        OPENING CAST:
        - 7 or 11: Odin's Favour -- warrior WINS!
        - 2, 3 or 12: Curse of the Norns -- warrior LOSES!
        - Other: that number becomes THE MARKED RUNE.
```

```
        RUNE PHASE: hit the rune = WIN, roll 7 = Ragnarök = LOSE.

        REQUIRED FORMAT:
        RUNES: [X, Y]
        TOTAL: [sum]
        DESTIN: [what happened]

        Respond in French.
        """)
    String play(@UserMessage String playerAction);
}
```

💡 **TOOLPROVIDERNAME**

`toolProviderName = "mcp"` connecte cet agent au bean `McpToolProvider`, qui découvre automatiquement les outils de lancer de runes depuis le serveur MCP externe.

2 Configurer MCP via les propriétés

Ouvrez `microprofile-config.properties` et décommentez la configuration MCP :

```
# Chat Model
# ---- Option A : Mistral AI (distant) ----
dev.langchain4j.cdi.plugin.mistral.class=dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.mistral.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.mistral.config.model-name=mistral-small-latest

# MCP Transport (Streamable HTTP vers le serveur MCP de dés)
dev.langchain4j.cdi.plugin.ssetransport.class=\
    dev.langchain4j.mcp.client.transport.http.StreamableHttpMcpTransport
dev.langchain4j.cdi.plugin.ssetransport.config.url=http://localhost:8090/mcp
dev.langchain4j.cdi.plugin.ssetransport.config.logRequests=true
dev.langchain4j.cdi.plugin.ssetransport.config.logResponses=true

# MCP Client
dev.langchain4j.cdi.plugin.mcpclient.class=dev.langchain4j.mcp.client.DefaultMcpClient
dev.langchain4j.cdi.plugin.mcpclient.config.transport=lookup:@ssetransport

# MCP Tool Provider (nommé "mcp" pour @RegisterAIService)
dev.langchain4j.cdi.plugin.mcp.class=dev.langchain4j.mcp.McpToolProvider
dev.langchain4j.cdi.plugin.mcp.config.mcpClients=lookup:@mcpclient
```

🏠 **OLLAMA LOCAL**

Même principe : commentez l'**Option A** du Chat Model et décommentez l'**Option B**. La config MCP (transport, client, tool provider) reste identique.

💡 **LE PATTERN LOOKUP:**

`lookup:@ssetransport` référence un autre bean CDI par son nom de configuration. C'est ainsi que vous câblez les beans entre eux uniquement via la configuration — aucun code Java nécessaire pour la plomberie.

3 Câbler le point d'accès REST

Ouvrez `GameResource.java` :

```
@Path("/game")
@ApplicationScoped
public class GameResource {

    @Inject
    HnefataflJarlAI gameMaster;

    @POST
    @Path("/play")
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String play(String playerAction) {
        return gameMaster.play(playerAction);
    }

    @GET
    @Path("/start")
    @Produces(MediaType.TEXT_PLAIN)
    public String start() {
        return gameMaster.play(
            "Salut ! Je suis prêt pour une partie de Hnefatafl !");
    }
}
```

✅ Tester et valider l'exercice 3

```
# Assurez-vous que le serveur MCP tourne sur le port 8090 !

# Déployer
cd demo-project/demo-3-mcp/base
mvn clean wildfly:dev

# Démarrer une partie
curl http://localhost:8080/demo-3/api/game/start

# Lancer les dés
curl -X POST -H "Content-Type: text/plain" \
  -d "Lance les runes !" \
  http://localhost:8080/demo-3/api/game/play

# Ouvrir l'interface web
# http://localhost:8080/demo-3/
```

- ✓ L'agent IA appelle l'outil `roll` du serveur MCP via Streamable HTTP
- ✓ Les résultats des runes sont retournés au format RUNES: [X, Y]
- ✓ Ragnar le Skald annonce les victoires, malédictions et runes marquées dans le style viking
- ✓ L'interface web affiche des dés visuels avec des points animés

> ⚔️ **FÉLICITATIONS !**
>
> Vous avez construit trois applications propulsées par l'IA avec LangChain4j-CDI : un skald viking, un guide d'expéditions résilient, et un jeu de Hnefatafl intégré via MCP. Le tout avec des patterns Jakarta EE standards.

Exercice 4 : Guardrails — Garde-fous pour votre agent

Message clé : « Les guardrails sont des beans CDI — ils bénéficient de l'injection de dépendances comme tous vos composants »

📌 **OBJECTIF**

Ajoutez quatre guardrails autour de votre skald viking : deux sur l'entrée (valider la requête avant d'appeler le LLM) et deux sur la sortie (valider la réponse avant de la renvoyer). L'agent n'accepte que le français et refuse toute mention de races fantastiques.

```
cd demo-project/demo-4-guardrails/base
# Ouvrir ce répertoire dans votre IDE
```

Concept : InputGuardrail vs OutputGuardrail

InputGuardrail

S'exécute **avant** d'appeler le LLM. Si le guardrail échoue, le LLM n'est jamais appelé — économie de temps et de tokens.

```
InputGuardrailResult validate(
    UserMessage userMessage)
```

OutputGuardrail

S'exécute **après** la réponse du LLM. Si le guardrail échoue, une exception est levée — le code appelant doit la gérer.

```
OutputGuardrailResult validate(
    AiMessage responseFromLLM)
```

Fichiers clés

FICHIER	RÔLE	TODO
`SkaldAssistant.java`	Interface du service IA	Ajouter `@RegisterAIService` avec les noms de guardrails
`FrenchInputGuardrail.java`	Vérifie que la requête est en français	Nommer, initialiser, implémenter
`FrenchOutputGuardrail.java`	Vérifie que le chant est en français	Nommer, initialiser, implémenter
`NoFantasyRacesInputGuardrail.java`	Rejette les mentions de nains/elfes en entrée	Nommer, implémenter

FICHIER	RÔLE	TODO
`NoFantasyRacesOutputGuardrail.java`	Rejette les mentions de nains/elfes en sortie	Nommer, implémenter
`ChatResource.java`	Point d'accès REST (déjà câblé)	Rien — gère déjà les exceptions guardrail

⚠ PRÉREQUIS

Cet exercice utilise **Mistral AI** (comme l'exercice 1). Assurez-vous que `MISTRAL_API_KEY` est exporté dans votre terminal.

1 Nommer les guardrails avec `@Named`

Les guardrails sont des beans CDI référencés par leur nom dans `@RegisterAIService`. Ajoutez `@Named` sur chaque classe de guardrail.

FrenchInputGuardrail.java

```
import jakarta.inject.Named;

@ApplicationScoped
@Named("french-input")   // ← Ajouter cette annotation
public class FrenchInputGuardrail implements InputGuardrail {
    // ...
}
```

FrenchOutputGuardrail.java

```
@ApplicationScoped
@Named("french-output")
public class FrenchOutputGuardrail implements OutputGuardrail {
    // ...
}
```

NoFantasyRacesInputGuardrail.java

```
@ApplicationScoped
@Named("fantasy-input")
```

```
public class NoFantasyRacesInputGuardrail implements InputGuardrail {
    // ...
}
```

NoFantasyRacesOutputGuardrail.java

```
@ApplicationScoped
@Named("fantasy-output")
public class NoFantasyRacesOutputGuardrail implements OutputGuardrail {
    // ...
}
```

> 💡 **POURQUOI @NAMED ?**
>
> LangChain4j-CDI résout les guardrails par leur nom CDI au moment du démarrage. Le nom dans `@Named("...")` doit correspondre exactement aux chaînes dans `inputGuardrailNames` et `outputGuardrailNames`.

2 Initialiser le détecteur de langue (Apache Tika)

Les guardrails de langue (`FrenchInputGuardrail` et `FrenchOutputGuardrail`) utilisent Apache Tika pour détecter automatiquement la langue d'un texte. Initialisez le détecteur dans `@PostConstruct` :

```
import org.apache.tika.langdetect.optimaize.OptimaizeLangDetector;
import org.apache.tika.language.detect.LanguageDetector;

@ApplicationScoped
@Named("french-input")
public class FrenchInputGuardrail implements InputGuardrail {

    private LanguageDetector detector;

    @PostConstruct
    void init() {
        detector = new OptimaizeLangDetector().loadModels();
        //        ↑ Charge les modèles de détection de langue depuis le classpath
    }

    // ...
}
```

Faites de même dans `FrenchOutputGuardrail` .

> 💡 **THREAD-SAFETY**
>
> Le bean est `@ApplicationScoped` : une seule instance partagée entre toutes les requêtes. Le `LanguageDetector` d'Apache Tika n'est **pas thread-safe** — vous devrez utiliser `synchronized (detector) { ... }` lors de la détection.

3 Implémenter la méthode `validate()`

FrenchInputGuardrail — accepter uniquement le français

```
@Override
public InputGuardrailResult validate(UserMessage userMessage) {
    String text = userMessage.singleText();
    LanguageResult result;
    synchronized (detector) {
        result = detector.detect(text);
    }
    // Benefit of the doubt : si Tika n'est pas sûr (texte trop court), on laisse passer
    if (result.isReasonablyCertain() && !"fr".equals(result.getLanguage())) {
        return failure("Seul le français est accepté ! Parlez français pour invoquer le skal
    }
    return success();
}
```

FrenchOutputGuardrail — vérifier que le chant est en français

```
@Override
public OutputGuardrailResult validate(AiMessage responseFromLLM) {
    String text = responseFromLLM.text();
    LanguageResult result;
    synchronized (detector) {
        result = detector.detect(text);
    }
    // Pour la sortie, on est strict : si pas certain ou pas français → échec
    if (!result.isReasonablyCertain() || !"fr".equals(result.getLanguage())) {
        return failure("Le chant doit être en français ! Le skald doit chanter dans la langu
    }
    return success();
}
```

💡 **DIFFÉRENCE ENTRÉE / SORTIE**

Pour l'**entrée** : on applique le *benefit of the doubt* — un seul mot comme « Odin » n'est pas détectable avec certitude, donc on accepte. Pour la **sortie** : le chant est long, Tika est fiable — on rejette si non certain ou si ce n'est pas du français.

NoFantasyRacesInputGuardrail — rejeter les mots interdits en entrée

```
private static final Set<String> FORBIDDEN_WORDS =
    Set.of("nain", "nains", "elfe", "elf", "elfes");

@Override
public InputGuardrailResult validate(UserMessage userMessage) {
    String text = userMessage.singleText().toLowerCase();
    for (String word : FORBIDDEN_WORDS) {
        if (text.contains(word)) {
            return failure("Les Vikings ne connaissent ni les nains ni les elfes ! "
                           + "Gardez votre requête purement nordique.");
        }
    }
    return success();
}
```

NoFantasyRacesOutputGuardrail — rejeter les mots interdits en sortie

```
@Override
public OutputGuardrailResult validate(AiMessage responseFromLLM) {
    String text = responseFromLLM.text().toLowerCase();
    for (String word : FORBIDDEN_WORDS) {
        if (text.contains(word)) {
            return failure("Le chant du skald ne doit pas évoquer les nains ni les elfes ! "
                           + "C'est une saga viking pure.");
        }
    }
    return success();
}
```

4 Câbler les guardrails dans `@RegisterAIService`

Ouvrez `SkaldAssistant.java` et décommentez / remplacez le TODO par l'annotation complète :

```
import dev.langchain4j.cdi.RegisterAIService;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;

@RegisterAIService(
    chatModelName        = "my-model",
    inputGuardrailNames  = {"french-input",  "fantasy-input"},
    outputGuardrailNames = {"french-output", "fantasy-output"})
public interface SkaldAssistant {

    @SystemMessage("""
        Tu es un skald viking qui compose des chants épiques en FRANÇAIS.
        Le chant doit célébrer les thèmes nordiques héroïques : batailles, raids,
        Valhalla, Thor, Odin, drakkars.
        Format : 3-4 couplets avec un refrain puissant.
        Ton : épique, héroïque, guerrier.
        """)
    String composeSong(@UserMessage String request);
}
```

💡 ORDRE D'EXÉCUTION

Les guardrails sont exécutés dans l'ordre des tableaux. Ici, `french-input` est évalué avant `fantasy-input`. Si le premier échoue, le second n'est pas évalué. Le LLM n'est appelé que si **tous** les guardrails d'entrée passent.

Déployer

```
mvn clean wildfly:dev
```

⚠ CONFIGURATION DÉJÀ FOURNIE

Le fichier `microprofile-config.properties` est déjà configuré avec le modèle Mistral AI. Assurez-vous que `MISTRAL_API_KEY` est défini dans votre environnement.

✓ Tester et valider l'exercice 4

Via curl

```
# ✅ Requête valide en français
curl -X POST -H "Content-Type: text/plain" \
  -d "Compose un chant sur la gloire d'Odin et les batailles vikings" \
  http://localhost:8080/demo-4/api/song

# ❌ InputGuardrail langue : requête en anglais rejetée
curl -X POST -H "Content-Type: text/plain" \
  -d "Sing me a heroic Viking song about Odin" \
  http://localhost:8080/demo-4/api/song

# ❌ InputGuardrail races : mention de nains rejetée
curl -X POST -H "Content-Type: text/plain" \
  -d "Compose un chant sur les nains qui forgeaient des armes vikings" \
  http://localhost:8080/demo-4/api/song

# Vérification santé
curl http://localhost:8080/demo-4/api/song/health
```

Via l'interface web

Ouvrez http://localhost:8080/demo-4/ pour accéder à l'interface interactive qui affiche les guardrails actifs et leurs messages d'erreur.

- ✓ Une requête en français sur un thème nordique produit un chant épique
- ✓ Une requête en anglais retourne *« Seul le français est accepté ! »*
- ✓ Une requête mentionnant des nains ou des elfes est rejetée
- ✓ Un seul mot ambigu comme « Odin » passe (benefit of the doubt)
- ✓ Le code HTTP 400 est retourné avec le message du guardrail qui a échoué

💡 **BLOQUÉ ?**

Passez à la solution : `cd ../solution && mvn clean wildfly:dev`

🎉 **FÉLICITATIONS !**

Vous avez construit quatre applications alimentées par l'IA avec LangChain4j-CDI. Passez à l'exercice 5 pour découvrir l'orchestration multi-agents avec le protocole A2A !

Exercice 5 : A2A — Orchestration multi-agents

Message clé : « MCP connecte les agents aux outils. A2A connecte les agents *entre eux* — les microservices version IA »

📌 **OBJECTIF**

Construisez trois services WildFly indépendants communicant via le protocole A2A (Agent-to-Agent) : un **Creative Writer** qui génère des histoires, un **Style Scorer** qui évalue le style, et un **Orchestrateur** qui coordonne le tout avec une boucle de révision automatique.

```
cd demo-project/demo-5-a2a/base
# Ouvrir ce répertoire dans votre IDE
```

Concept : MCP vs A2A

MCP (Model Context Protocol)

Un agent appelle des **outils** externes (bases de données, APIs, fichiers). Communication agent → outil.

```
Agent → MCP Server → Tool
```

A2A (Agent-to-Agent)

Un agent **délègue** à d'autres agents spécialisés, chacun sur son propre serveur. Communication agent ↔ agent.

```
Orchestrateur ↔ Agent A2A ↔ Agent A2A
```

Architecture de Story Forge

L'application **Story Forge** orchestre trois services :

1. **Creative Writer** (port 8080) — Forge une saga nordique courte sur un sujet donné
2. **Style Scorer** (port 8081) — Évalue l'adéquation de la saga avec un style (score 0.0–1.0)
3. **Orchestrateur** (port 8082) — Coordonne le pipeline : écriture → boucle (scorer + éditeur local) jusqu'à score ≥ 0.8

Fichiers clés

a2a-creative-writer

FICHIER	RÔLE	TODO
`CreativeWriter.java`	Service IA LangChain4j	Ajouter `@RegisterAIService` et `@UserMessage`
`CreativeWriterAgentCardProducer.java`	Décrit l'agent au réseau A2A	Construire l'`AgentCard`
`CreativeWriterExecutorProducer.java`	Reçoit et traite les requêtes A2A	Implémenter `execute()`

a2a-style-scorer

FICHIER	RÔLE	TODO
`StyleScorer.java`	Service IA LangChain4j	Ajouter `@RegisterAIService` et `@UserMessage`
`StyleScorerAgentCardProducer.java`	Décrit l'agent au réseau A2A	Construire l'`AgentCard`
`StyleScorerExecutorProducer.java`	Reçoit et traite les requêtes A2A	Implémenter `execute()`

a2a-orchestrator

FICHIER	RÔLE	TODO
`Agents.java`	Interfaces des agents du pipeline	Ajouter `@Agent`, `@UserMessage`, `AgenticScopeAccess`
`OrchestratorService.java`	Câble le pipeline agentic	Créer les builders `a2a`, `loop`, `sequence`
`StyledWriterEndpoint.java`	Point d'accès REST (déjà câblé)	Rien — déjà implémenté

⚠ PRÉREQUIS

Cet exercice utilise **Ollama** avec le modèle `qwen2.5:7b`. Assurez-vous qu'Ollama tourne (`ollama serve`) et que le modèle est téléchargé (`ollama pull qwen2.5:7b`).

1 Implémenter le service IA `CreativeWriter`

Ouvrez `a2a-creative-writer/src/main/java/com/example/demo5/writer/CreativeWriter.java` et complétez l'interface :

```
package com.example.demo5.writer;

import dev.langchain4j.cdi.spi.RegisterAIService;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.service.V;
import jakarta.enterprise.context.ApplicationScoped;

@RegisterAIService(chatModelName = "ollama", scope = ApplicationScoped.class)
public interface CreativeWriter {

    @UserMessage("""
                You are a Norse skald, a Viking bard who crafts mighty sagas of glory, battl
                Forge a short saga no more than 3 sentences around the given topic, in the s
                Return only the saga and nothing else.
                The topic is {{topic}}.
                """)
    String generateStory(@V("topic") String topic);
}
```

> 💡 **@REGISTERAISERVICE**
>
> Comme dans les exercices précédents, `@RegisterAIService` transforme l'interface en bean CDI injectable. Le `chatModelName = "ollama"` référence la configuration dans `microprofile-config.properties`.

2 Implémenter le service IA `StyleScorer`

Ouvrez `a2a-style-scorer/src/main/java/com/example/demo5/scorer/StyleScorer.java` et complétez l'interface :

```
package com.example.demo5.scorer;

import dev.langchain4j.cdi.spi.RegisterAIService;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.service.V;
import jakarta.enterprise.context.ApplicationScoped;
```

```
@RegisterAIService(chatModelName = "ollama", scope = ApplicationScoped.class)
public interface StyleScorer {

    @UserMessage("""
            You are a seasoned Norse skald elder who judges sagas by the fire of a longhouse
            Give a score between 0.0 and 1.0 for the following saga
            based on how well it captures the '{{style}}' style.
            Return only the score and nothing else.

            The saga is: "{{story}}"
            """)
    double scoreStyle(@V("story") String story, @V("style") String style);
}
```

💡 **TYPE DE RETOUR** `DOUBLE`

LangChain4j parse automatiquement la réponse du LLM en `double`. Le prompt demande explicitement un score numérique pour faciliter le parsing.

3 Créer l'AgentCard et l'AgentExecutor

Chaque agent A2A a besoin de deux composants CDI : une **AgentCard** (qui décrit l'agent au réseau) et un **AgentExecutor** (qui traite les requêtes).

AgentCard — Creative Writer

Ouvrez `CreativeWriterAgentCardProducer.java` et implémentez `agentCard()` :

```
@Produces
@PublicAgentCard
public AgentCard agentCard() {
    return new AgentCard.Builder()
            .name("Creative Writer")
            .description("Generate a Norse saga based on the given topic")
            .url(serverUrl)
            .version("1.0.0")
            .protocolVersion("1.0.0")
            .capabilities(new AgentCapabilities.Builder()
                    .streaming(true)
                    .pushNotifications(false)
                    .stateTransitionHistory(false)
                    .build())
            .defaultInputModes(Collections.singletonList("text"))
```

```
            .defaultOutputModes(Collections.singletonList("text"))
            .skills(Collections.singletonList(new AgentSkill.Builder()
                    .id("creative_writer")
                    .name("Creative Writer")
                    .description("Generate a Norse saga based on the given topic")
                    .tags(Collections.singletonList("writing"))
                    .build()))
            .protocolVersion("0.3.0")
            .preferredTransport(TransportProtocol.JSONRPC.asString())
            .additionalInterfaces(List.of(
                    new AgentInterface(TransportProtocol.JSONRPC.asString(), serverUrl)))
            .build();
}
```

AgentExecutor — Creative Writer

Ouvrez `CreativeWriterExecutorProducer.java` et décommentez les étapes 1–4 dans `execute()` :

```
@Override
public void execute(RequestContext context, EventQueue eventQueue) throws JSONRPCError {
    TaskUpdater updater = new TaskUpdater(context, eventQueue);

    // ÉTAPE 1 : Soumettre et démarrer
    if (context.getTask() == null) { updater.submit(); }
    updater.startWork();

    // ÉTAPE 2 : Extraire le texte du message A2A
    String userMessage = extractTextFromMessage(context.getMessage());

    // ÉTAPE 3 : Appeler le service IA
    String response = creativeWriter.generateStory(userMessage);

    // ÉTAPE 4 : Répondre et compléter
    TextPart responsePart = new TextPart(response, null);
    updater.addArtifact(List.of(responsePart), null, null, null);
    updater.complete();
}
```

Répéter pour le Style Scorer

Appliquez le même pattern dans `StyleScorerAgentCardProducer.java` (skill id : `"style_scorer"`) et `StyleScorerExecutorProducer.java` (extraire deux arguments : story et style).

> 💡 **PATTERN SERVEUR A2A**

> Un serveur A2A Jakarta EE se résume à trois composants CDI : le `@RegisterAIService` (logique IA), un `@Produces @PublicAgentCard` (description), et un `@Produces AgentExecutor` (pont entre A2A et le service IA).

4 Câbler l'orchestrateur agentic

Définir les interfaces d'agents

Ouvrez `a2a-orchestrator/src/.../orchestrator/Agents.java` et ajoutez les annotations :

```
import dev.langchain4j.agentic.Agent;
import dev.langchain4j.agentic.scope.AgenticScopeAccess;
import dev.langchain4j.agentic.scope.ResultWithAgenticScope;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.service.V;

public class Agents {

    public interface A2ACreativeWriter {
        @Agent(description = "Generate a Norse saga based on the given topic",
               outputKey = "story")
        String generateStory(@V("topic") String topic);
    }

    public interface A2AStyleScorer {
        @Agent(description = "Score a saga based on how well it captures a given style",
               outputKey = "score")
        double scoreStyle(@V("story") String story, @V("style") String style);
    }

    public interface StyleEditor {
        @UserMessage("""
                You are a master Norse skald who shapes and tempers sagas like a smith forge
                Rewrite the following saga to better honor and embody the {{style}} style.
                Return only the saga and nothing else.
                The saga is "{{story}}".
                """)
        @Agent(description = "Reforge a saga to better capture a given style",
               outputKey = "story")
        String editStory(@V("story") String story, @V("style") String style);
    }

    public interface StyleReviewLoop {
        @Agent("Judge the saga by the standards of the specified style, as a Viking elder wo
```

```
        String scoreAndReview(@V("story") String story, @V("style") String style);
    }

    public interface StyledWriter extends AgenticScopeAccess {
        @Agent
        ResultWithAgenticScope<String> writeStoryWithStyle(
                @V("topic") String topic, @V("style") String style);
    }
}
```

Câbler le pipeline dans `OrchestratorService.java`

Décommentez les 6 étapes dans `init()` :

```
@PostConstruct
void init() {
    // ÉTAPE 1 : ChatModel local pour le StyleEditor
    ChatModel chatModel = OllamaChatModel.builder()
            .baseUrl(ollamaBaseUrl).modelName(ollamaModelName)
            .timeout(Duration.ofMinutes(10)).temperature(0.0)
            .logRequests(true).logResponses(true).build();

    // ÉTAPE 2 : Agent A2A distant — Creative Writer
    A2ACreativeWriter creativeWriter = AgenticServices
            .a2aBuilder(creativeWriterUrl, A2ACreativeWriter.class)
            .outputKey("story").build();

    // ÉTAPE 3 : Agent A2A distant — Style Scorer
    A2AStyleScorer styleScorer = AgenticServices
            .a2aBuilder(styleScorerUrl, A2AStyleScorer.class)
            .outputKey("score").build();

    // ÉTAPE 4 : Agent local — Style Editor
    StyleEditor styleEditor = AgenticServices.agentBuilder(StyleEditor.class)
            .chatModel(chatModel).outputKey("story").build();

    // ÉTAPE 5 : Boucle de révision (max 5 itérations, sort si score ≥ 0.8)
    UntypedAgent styleReviewLoop = AgenticServices.loopBuilder()
            .subAgents(styleScorer, styleEditor)
            .maxIterations(5)
            .exitCondition(scope -> scope.readState("score", 0.0) ≥ 0.8)
            .build();

    // ÉTAPE 6 : Séquence complète
    styledWriter = AgenticServices.sequenceBuilder(StyledWriter.class)
            .subAgents(creativeWriter, styleReviewLoop)
            .outputKey("story").build();
}
```

💡 **LANGCHAIN4J AGENTIC**

Trois builders pour orchestrer des agents :

- `a2aBuilder()` — proxy vers un agent A2A distant
- `loopBuilder()` — boucle avec condition de sortie
- `sequenceBuilder()` — pipeline séquentiel

L' `outputKey` permet de partager des données entre agents via le scope agentic.

Démarrer les trois services

Chaque service tourne sur un port différent. Ouvrez **trois terminaux** :

```
# Terminal 1 : Creative Writer (port 8080)
cd demo-project/demo-5-a2a/base/a2a-creative-writer
mvn clean wildfly:dev

# Terminal 2 : Style Scorer (port 8081)
cd demo-project/demo-5-a2a/base/a2a-style-scorer
mvn clean wildfly:dev -Djboss.socket.binding.port-offset=1

# Terminal 3 : Orchestrateur (port 8082)
cd demo-project/demo-5-a2a/base/a2a-orchestrator
mvn clean wildfly:dev -Djboss.socket.binding.port-offset=2
```

⚠ **ORDRE DE DÉMARRAGE**

Démarrez le Creative Writer et le Style Scorer **avant** l'Orchestrateur. L'orchestrateur se connecte aux agents A2A au démarrage.

✓ Tester et valider l'exercice 5

Via curl

```
# Forger une saga avec un style
curl "http://localhost:8082/api/styled-story?topic=Erik+le+Rouge+traverse+les+mers&style=epi

# Autre exemple
curl "http://localhost:8082/api/styled-story?topic=la+bataille+de+Ragnarok&style=tragique"
```

Via l'interface web

Ouvrez http://localhost:8082 pour accéder à l'interface Story Forge. Saisissez un sujet et un style, puis observez la génération.

- ✓ Les trois services WildFly démarrent sans erreur (ports 8080, 8081, 8082)
- ✓ L'interface web Story Forge s'affiche sur le port 8082
- ✓ Une requête produit une saga avec un score de style
- ✓ Les logs montrent la boucle de révision (scorer → editor → scorer...)
- ✓ Le score final est ≥ 0.8 (ou max 5 itérations atteintes)

💡 **BLOQUÉ ?**

Passez à la solution : utilisez les modules sous `demo-5-a2a/solution/` au lieu de `base/`.

🎉 **FÉLICITATIONS !**

Vous avez construit cinq applications alimentées par l'IA avec LangChain4j-CDI : un skald viking, un guide d'expéditions résilient, un jeu de dés avec MCP, un skald avec guardrails, et maintenant un pipeline multi-agents via A2A. Tous avec les patterns Jakarta EE que vous connaissez déjà.

Et après ?

Modules LangChain4j-CDI

MODULE	ARTEFACT	UTILITÉ
core	langchain4j-cdi-core	API + annotations
portable	langchain4j-cdi-portable-ext	WildFly, Payara, GlassFish, Liberty
build	langchain4j-cdi-build-compatible-ext	Quarkus, Helidon
config	langchain4j-cdi-config	MicroProfile Config SPI

MODULE	ARTEFACT	UTILITÉ
FT	langchain4j-cdi-fault-tolerance	Retry, Timeout, CircuitBreaker
telemetry	langchain4j-cdi-telemetry	Spans et métriques OpenTelemetry

Ressources

- LangChain4j-CDI on GitHub
- LangChain4j Documentation
- MCP Specification
- WildFly Documentation

Dépannage

PROBLÈME	CAUSE	SOLUTION
`ClassNotFoundException: ...faulttolerance.Retry`	Couches Galleon manquantes	Ajouter `microprofile-fault-tolerance` dans `<layers>`
Propriétés ignorées silencieusement	Préfixe `.config.` manquant	Utiliser `...plugin.X.config.prop=val`
La mémoire se réinitialise à chaque message	`get()` crée une nouvelle instance	Utiliser `computeIfAbsent()` dans le provider
`ChatMessage.text()` ne compile pas	`ChatMessage` n'a pas de `text()`	Pattern-match : `SystemMessage`, `UserMessage`, etc.
404 sur la racine du contexte	Pas de `<name>` dans le plugin wildfly	Ajouter `<name>demo-N</name>`
Le serveur MCP ne répond pas	Le serveur ne tourne pas sur le port 8090	Démarrer le serveur MCP d'abord
`MISTRAL_API_KEY` introuvable	Variable d'environnement non définie	`export MISTRAL_API_KEY=your-key`
Guardrail jamais déclenché	`@Named` manquant ou nom différent	Vérifier que le nom dans `@Named("...")` correspond exactement à

PROBLÈME	CAUSE	SOLUTION
		`inputGuardrailNames`
`NullPointerException` dans le guardrail	Détecteur non initialisé	Vérifier que `@PostConstruct` appelle bien `new OptimaizeLangDetector().loadModels()`
Texte français rejeté par `french-input`	Texte trop court pour Tika	Normal — la logique `isReasonablyCertain()` donne le benefit of the doubt aux textes ambigus
Réponse 400 sans message lisible	Exception guardrail non parsée	Le `ChatResource` extrait le message avec `extractGuardrailMessage()` — vérifier l'implémentation
L'orchestrateur ne démarre pas	Agents A2A non accessibles	Démarrer Creative Writer (8080) et Style Scorer (8081) **avant** l'Orchestrateur (8082)
`Connection refused` vers l'agent A2A	Port-offset manquant	Utiliser `-Djboss.socket.binding.port-offset=1` pour le scorer, `=2` pour l'orchestrateur
Score toujours < 0.8 après 5 itérations	Modèle trop petit ou style vague	Normal — la boucle s'arrête après `maxIterations(5)`. Essayez un style plus simple.
`AgentCard` non trouvée	`@PublicAgentCard` manquant	Vérifier que le producteur CDI est annoté `@Produces @PublicAgentCard`

⚠ SECOURS D'URGENCE

Si quelque chose ne va pas, passez au module solution :

```
cd demo-N-xxx/solution && mvn clean wildfly:dev
# Exemple pour l'exercice 4 :
cd demo-4-guardrails/solution && mvn clean wildfly:dev
# Exemple pour l'exercice 5 (3 terminaux) :
cd demo-5-a2a/solution/a2a-creative-writer && mvn clean wildfly:dev
cd demo-5-a2a/solution/a2a-style-scorer && mvn clean wildfly:dev -Djboss.socket.binding.port-
cd demo-5-a2a/solution/a2a-orchestrator && mvn clean wildfly:dev -Djboss.socket.binding.port-
```

Atelier LangChain4j-CDI — Devoxx France 2026 — GitHub — Licence : Apache 2.0

Par **Yann Blazart** & **Emmanuel Hugonnet**